

Full-Text Search in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [FTS Architecture Overview](#)
 3. [tsvector — The Document Representation](#)
 4. [tsquery — The Search Query](#)
 5. [Conversion Functions](#)
 6. [Search Operators and Logic](#)
 7. [Ranking: ts_rank and ts_rank_cd](#)
 8. [Highlighting with ts_headline](#)
 9. [GIN and GiST Indexes for FTS](#)
 10. [Multi-column Search](#)
 11. [Custom Dictionaries and Configurations](#)
 12. [Real-World Schema Patterns](#)
 13. [Production Monitoring Queries](#)
 14. [Common Mistakes](#)
 15. [Best Practices](#)
 16. [Performance Considerations](#)
 17. [Interview Questions & Answers](#)
 18. [Exercises and Solutions](#)
 19. [Cross-References](#)
-

Learning Objectives

By the end of this module you will be able to:

- Explain how PostgreSQL's FTS pipeline transforms text to searchable vectors
 - Use all four query-building functions and understand when to choose each
 - Implement ranking and highlighting in a production search endpoint
 - Choose between GIN and GiST indexes based on workload characteristics
 - Build a multi-language, multi-column full-text search system
 - Diagnose and optimize FTS query performance
-

FTS Architecture Overview

Full-Text Search Pipeline:

Raw Text: "PostgreSQL is the World's most advanced open-source database"

|



```
to_tsvector('english', text)
```

```
Step 1: Parse → tokens
```

```
Step 2: Normalize (lowercase, stemming)
```

```
Step 3: Remove stop words (is, the, most, s)
```

```
Step 4: Assign lexemes with positions
```



```
-- Basic tsvector creation
SELECT 'fat cats ate fat rats'::tsvector;
-- 'ate' 'cats' 'fat' 'rats' (sorted, deduplicated, no positions)

-- With positions
SELECT to_tsvector('english', 'fat cats ate fat rats');
-- 'ate':3 'cat':2 'fat':1,4 'rat':5

-- Manual tsvector with weights
SELECT setweight(to_tsvector('english', 'PostgreSQL Tutorial'), 'A') ||
       setweight(to_tsvector('english', 'Learn PostgreSQL from scratch'), 'B');
-- 'learn':4B 'postgresql':1A,3B 'scratch':6B 'tutori':2A
```

Weight classes

Weight	Meaning (convention)	Boost
A	Title / heading	Highest
B	Subtitle / abstract	High
C	Body text	Normal
D	Footer / metadata	Lowest (default)

```
-- Weighted document from multiple columns
SELECT setweight(to_tsvector('english', title), 'A') ||
       setweight(to_tsvector('english', COALESCE(subtitle, '')), 'B') ||
       setweight(to_tsvector('english', body), 'C')
AS document_vector
FROM articles;
```

tsquery — The Search Query

A `tsquery` represents a search expression of lexemes connected by Boolean operators.

```
-- Basic tsquery
SELECT 'postgresql & database'::tsquery;
-- 'postgresql' & 'databas' ← note: stemmed!

-- OR search
SELECT 'postgresql | mysql'::tsquery;

-- NOT
SELECT 'database & !oracle'::tsquery;

-- Phrase search (proximity, PostgreSQL 9.6+)
SELECT 'postgresql <-> database'::tsquery; -- adjacent
SELECT 'postgresql <2> guide'::tsquery;     -- within 2 words

-- Prefix matching
SELECT 'postg:*'::tsquery; -- matches postgresql, postgres, postgis, etc.

-- Parentheses for grouping
SELECT '(postgresql | mysql) & performance'::tsquery;
```

Conversion Functions

to_tsvector — Parse and normalize text

```
-- Syntax: to_tsvector([config,] text)
SELECT to_tsvector('english', 'The quick brown fox jumps over the lazy dog');
-- 'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2

SELECT to_tsvector('simple', 'Hello World'); -- no stemming
-- 'hello':1 'world':2

SELECT to_tsvector('french', 'Les chats mangent des souris');
-- 'chat':2 'mang':3 'souris':5

-- From a column
SELECT id, to_tsvector('english', body) FROM articles LIMIT 3;
```

to_tsquery — Parse query with normalization

```
-- Syntax: to_tsquery([config,] text)
SELECT to_tsquery('english', 'running & dogs');
-- 'run' & 'dog' ← stemmed

SELECT to_tsquery('english', 'postgreSQL');
```

```
-- 'postgresql'

-- Errors on invalid syntax:
SELECT to_tsquery('hello world'); -- ERROR: syntax error (needs &, |, !)
```

plainto_tsquery — Natural language query (AND-based)

```
-- Converts a natural language phrase to AND query
SELECT plainto_tsquery('english', 'fast postgres database');
-- 'fast' & 'postgr' & 'databas'

-- Safe for user input – no syntax errors
SELECT plainto_tsquery('english', 'the quick brown fox');
-- 'quick' & 'brown' & 'fox' (stop words removed)
```

websearch_to_tsquery — Google-style syntax

```
-- Supports: quoted phrases, - for NOT, OR keyword
SELECT websearch_to_tsquery('english', 'postgres database -oracle');
-- 'postgr' & 'databas' & !'oracl'

SELECT websearch_to_tsquery('english', '"full text search" OR "text search"');
-- 'full' <-> 'text' <-> 'search' | 'text' <-> 'search'

SELECT websearch_to_tsquery('english', 'postgres OR mysql performance');
-- ( 'postgr' | 'mysql' ) & 'perform'

-- Best for user-facing search boxes (never errors, familiar syntax)
```

phraseto_tsquery — Exact phrase search

```
SELECT phraseto_tsquery('english', 'full text search');
-- 'full' <-> 'text' <-> 'search'

-- Finds documents where these words appear consecutively in order
```

Comparison summary

Function	Use case	User input safe	Stemming
to_tsquery	Programmatic, precise	No	Yes
plainto_tsquery	Simple natural language	Yes	Yes
websearch_to_tsquery	User search boxes	Yes	Yes
phraseto_tsquery	Exact phrase lookup	Yes	Yes

Search Operators and Logic

```
-- Setup: article search system
CREATE TABLE articles (
  id          SERIAL PRIMARY KEY,
  title       TEXT NOT NULL,
  subtitle    TEXT,
  body        TEXT NOT NULL,
  author      TEXT,
  language    TEXT DEFAULT 'english',
  published   BOOLEAN DEFAULT false,
  created_at  TIMESTAMPTZ DEFAULT now()
);

INSERT INTO articles (title, subtitle, body, author) VALUES
('PostgreSQL Performance Tuning',
 'A comprehensive guide to database optimization',
 'PostgreSQL offers many tools for performance tuning. Understanding query plans,
 indexes, and vacuum processes are fundamental. The EXPLAIN ANALYZE command reveals
 execution details including actual rows and timing.',
 'Alice Smith'),
('Full Text Search in PostgreSQL',
 'Building search features with native FTS',
 'PostgreSQL supports full text search through tsvector and tsquery types. GIN
 indexes make FTS queries extremely fast on large datasets. Configuration includes
 multiple languages and custom dictionaries.',
 'Bob Jones'),
('Database Design Patterns',
 'Normalization and beyond',
 'Relational database design requires careful consideration of normalization forms.
 Third normal form prevents update anomalies. Modern databases sometimes
 intentionally denormalize for performance.',
 'Carol White'),
('PostgreSQL Replication Guide',
 'Streaming and logical replication explained',
 'PostgreSQL streaming replication provides high availability and read scaling.
 Logical replication enables selective table replication and cross-version upgrades.
 Replication slots prevent WAL segment removal.',
 'Alice Smith'),
('Index Strategies for Large Databases',
 NULL,
 'B-tree indexes are the default and work well for equality and range queries. GIN
 indexes suit multi-value columns like arrays and JSONB. Partial indexes reduce index
 size by filtering rows.',
 'Dave Brown');

-- Add pre-computed tsvector column (best practice)
ALTER TABLE articles ADD COLUMN search_vector TSVECTOR;

UPDATE articles SET search_vector =
```

```

setweight(to_tsvector('english', title), 'A') ||
setweight(to_tsvector('english', COALESCE(subtitle, '')), 'B') ||
setweight(to_tsvector('english', body), 'C');

-- Simple FTS query
SELECT title FROM articles
WHERE search_vector @@ to_tsquery('english', 'postgresql & performance');

-- Using plainto_tsquery
SELECT title FROM articles
WHERE search_vector @@ plainto_tsquery('english', 'database replication');

-- Using websearch_to_tsquery (production recommended)
SELECT title FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', 'postgresql -oracle');

-- Phrase search
SELECT title FROM articles
WHERE search_vector @@ phraseto_tsquery('english', 'full text search');

-- Prefix matching
SELECT title FROM articles
WHERE search_vector @@ to_tsquery('english', 'replicat:');
-- Matches: replication, replicating, replicated

```

Ranking: ts_rank and ts_rank_cd

ts_rank — Frequency-based ranking

```

-- Syntax: ts_rank([weights,] vector, query [,normalization])
SELECT
    title,
    ts_rank(search_vector, websearch_to_tsquery('english', 'postgresql')) AS rank
FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', 'postgresql')
ORDER BY rank DESC;

```

ts_rank_cd — Cover density ranking

```

-- ts_rank_cd considers how close matches are to each other
SELECT
    title,
    ts_rank_cd(search_vector, websearch_to_tsquery('english', 'postgresql index'))
AS rank_cd
FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', 'postgresql index')
ORDER BY rank_cd DESC;

```

Normalization options (bitmask)

```
-- 0 = raw rank (default)
-- 1 = divide by 1 + log(document length)
-- 2 = divide by document length
-- 4 = divide by mean harmonic distance between extents
-- 8 = divide by unique words count
-- 16 = divide by 1 + log(unique words count)
-- 32 = divide by rank + 1

-- Normalize by document length (prevents long docs from dominating)
SELECT title,
       ts_rank(search_vector, query, 2) AS rank
FROM articles,
     websearch_to_tsquery('english', 'postgresql') AS query
WHERE search_vector @@ query
ORDER BY rank DESC;
```

Combining rank with other signals

```
-- Boost recent articles
SELECT
    id,
    title,
    ts_rank_cd(search_vector, query, 32) *
    EXP(-EXTRACT(EPOCH FROM (now() - created_at)) / (86400 * 30)) AS blended_score
FROM articles,
     websearch_to_tsquery('english', 'postgresql performance') AS query
WHERE search_vector @@ query
ORDER BY blended_score DESC;
```

Highlighting with ts_headline

`ts_headline` marks matching terms in the original text.

```
-- Basic headline
SELECT
    title,
    ts_headline(
        'english',
        body,
        websearch_to_tsquery('english', 'index performance'),
        'MaxWords=50, MinWords=20, ShortWord=3, MaxFragments=3'
    ) AS excerpt
FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', 'index performance');
```

Headline options

Option	Default	Description
StartSel		Highlight start tag
StopSel		Highlight stop tag
MaxWords	35	Maximum words in fragment
MinWords	15	Minimum words in fragment
ShortWord	3	Words shorter than this skipped for fragment boundaries
HighlightAll	false	Highlight all matches, not just fragments
MaxFragments	0 (unlimited)	Maximum number of fragments
FragmentDelimiter	" ... "	Delimiter between fragments

```
-- Custom highlighting for API response
SELECT
  id,
  ts_headline('english', title, query,
    'StartSel=<mark>, StopSel=</mark>, HighlightAll=true') AS title_h1,
  ts_headline('english', body, query,
    'StartSel=<mark>, StopSel=</mark>, MaxFragments=2, MaxWords=40') AS
body_excerpt
FROM articles,
  websearch_to_tsquery('english', 'postgresl replication') AS query
WHERE search_vector @@ query;
```

Warning: `ts_headline` is expensive — it re-parses the original text. Avoid calling it on large result sets; paginate first.

GIN and GiST Indexes for FTS

GIN vs GiST for Full-Text Search:

GIN Index

Inverted index
'databas' → {1,2,3}
'index' → {2,4,5}
'postgr' → {1,2,4}

Query speed: FAST
Build speed: SLOW
Index size: LARGE

GiST Index

Generalized search
tree (signatures)
Approximate match
faster to build

Query speed: SLOWER
Build speed: FAST
Index size: SMALL

False positives: NONE	False positives: YES (recheck needed)
Updates: Expensive	Updates: Cheap

```
-- GIN index (production default)
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vector);

-- GiST index (better for frequently updated tables)
CREATE INDEX idx_articles_fts_gist ON articles USING GIST (search_vector);

-- Inline to_tsvector (no stored column needed, but slower)
CREATE INDEX idx_articles_inline ON articles
USING GIN (to_tsvector('english', title || ' ' || body));

-- Check index usage
EXPLAIN (ANALYZE, BUFFERS)
SELECT title FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', 'postgresql');
-- Should show: Bitmap Index Scan on idx_articles_fts
```

GIN fast update

```
-- Disable fast update for consistent write performance
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vector)
WITH (fastupdate = off);

-- For bulk loads, re-enable after:
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vector)
WITH (fastupdate = on); -- default
-- Then VACUUM ANALYZE after bulk insert to flush pending list
```

Multi-column Search

```
-- Trigger to auto-update search_vector
CREATE OR REPLACE FUNCTION articles_search_vector_update()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.search_vector :=
        setweight(to_tsvector('english', COALESCE(NEW.title, '')), 'A') ||
        setweight(to_tsvector('english', COALESCE(NEW.subtitle, '')), 'B') ||
        setweight(to_tsvector('english', COALESCE(NEW.body, '')), 'C') ||
        setweight(to_tsvector('english', COALESCE(NEW.author, '')), 'D');
    RETURN NEW;
END;
$$;

CREATE TRIGGER articles_search_vector_trigger
BEFORE INSERT OR UPDATE ON articles
FOR EACH ROW EXECUTE FUNCTION articles_search_vector_update();
```

```
-- Now inserts/updates auto-maintain the search_vector
INSERT INTO articles (title, body, author)
VALUES ('New Article', 'Content here', 'Eve Green');
-- search_vector is automatically populated
```

Custom Dictionaries and Configurations

```
-- List available text search configurations
SELECT cfgname FROM pg_ts_config;

-- List available dictionaries
SELECT dictname FROM pg_ts_dict;

-- Create a custom configuration (English + custom stop words)
CREATE TEXT SEARCH CONFIGURATION my_english (COPY = english);

-- Create a synonym dictionary
CREATE TEXT SEARCH DICTIONARY my_synonyms (
    TEMPLATE = thesaurus,
    DictFile = mythesaurus,
    Dictionary = english_stem
);

-- Check how a configuration tokenizes text
SELECT * FROM ts_debug('english', 'PostgreSQL database performance tuning');
-- Shows: token, type, dictionary used, lexeme output

-- Test a configuration
SELECT to_tsvector('english', 'running databases are running fast');
-- 'databas':2 'fast':5 'run':1,4 (stemmed: running-run, databases-databas)
```

Real-World Schema Patterns

Pattern 1: Generated column for auto-maintenance

```
-- PostgreSQL 12+ generated columns
ALTER TABLE articles
ADD COLUMN search_vec TSVECTOR
GENERATED ALWAYS AS (
    setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', COALESCE(subtitle, '')), 'B') ||
    setweight(to_tsvector('english', body), 'C')
) STORED;

CREATE INDEX ON articles USING GIN (search_vec);
```

Pattern 2: Multi-language support

```
ALTER TABLE articles ADD COLUMN lang REGCONFIG DEFAULT 'english';

-- Store config per row
UPDATE articles SET lang = 'french' WHERE language_code = 'fr';
UPDATE articles SET lang = 'english' WHERE language_code = 'en';

-- Index using per-row config
CREATE INDEX idx_articles_multilang ON articles
USING GIN (to_tsvector(lang, title || ' ' || body));

-- Query
SELECT title FROM articles
WHERE to_tsvector(lang, title || ' ' || body) @@
      plainto_tsquery(lang::text, 'recherche texte');
```

Pattern 3: Search API with pagination and ranking

```
CREATE OR REPLACE FUNCTION search_articles(
    p_query    TEXT,
    p_page     INT DEFAULT 1,
    p_per_page INT DEFAULT 10
)
RETURNS TABLE (
    id          INT,
    title       TEXT,
    excerpt     TEXT,
    rank        FLOAT4,
    total_count BIGINT
) LANGUAGE sql STABLE AS $$
WITH results AS (
    SELECT
        a.id,
        a.title,
        a.body,
        ts_rank_cd(a.search_vector, q, 32) AS rank,
        COUNT(*) OVER () AS total_count
    FROM articles a,
         websearch_to_tsquery('english', p_query) AS q
    WHERE a.search_vector @@ q
          AND a.published = true
    ORDER BY rank DESC
    LIMIT p_per_page
    OFFSET (p_page - 1) * p_per_page
)
SELECT
    r.id,
    r.title,
    ts_headline('english', r.body, websearch_to_tsquery('english', p_query),
```

```

        'MaxFragments=2, MaxWords=40, MinWords=15') AS excerpt,
        r.rank,
        r.total_count
    FROM results r;
$$;

-- Usage
SELECT * FROM search_articles('postgresql performance', 1, 10);

```

Production Monitoring Queries

```

-- Find slow FTS queries
SELECT
    query,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS avg_ms,
    ROUND(total_exec_time::numeric, 2) AS total_ms
FROM pg_stat_statements
WHERE query ILIKE '%%@%'
    OR query ILIKE '%to_tsvector%'
ORDER BY mean_exec_time DESC
LIMIT 20;

-- Check FTS index health
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
WHERE indexdef LIKE '%GIN%'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Check for missing search vector updates (trigger health)
SELECT COUNT(*) AS rows_missing_vector
FROM articles
WHERE search_vector IS NULL;

-- GIN index pending list size (for fastupdate=on)
SELECT
    n.nspname || '.' || c.relname AS index_name,
    pg_size_pretty(pg_relation_size(c.oid)) AS index_size
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_index i ON i.indexrelid = c.oid

```

```
WHERE c.relam = (SELECT oid FROM pg_am WHERE amname = 'gin')
ORDER BY pg_relation_size(c.oid) DESC;
```

Common Mistakes

1. **Using `ts_headline` on unsorted large result sets** — It re-parses original text. Always paginate first, then highlight only the current page.
2. **Forgetting to index the `tsvector` column** — FTS without a GIN index is a sequential scan. Always `CREATE INDEX ... USING GIN (search_vector)`.
3. **Not handling NULL in concatenation** — `to_tsvector('english', title || body)` fails if `body` is NULL. Use `COALESCE(body, '')`.
4. **Searching with `LIKE '%keyword%'`** — This doesn't use FTS indexes. Use `@@` with proper `tsquery`.
5. **Forgetting to update the `tsvector` column** — If using a stored column without a trigger, it goes stale. Use triggers or generated columns.
6. **Using `to_tsquery` for user input** — User input like `hello world` (without `&`) causes syntax errors. Use `websearch_to_tsquery` for user-facing search.
7. **Hardcoding 'english' language** — For multi-language apps, use per-row language configuration stored in a `REGCONFIG` column.

Best Practices

1. **Stored computed column + trigger** — Pre-compute `tsvector` and store it. Re-computing on every query is expensive.
2. **Use `websearch_to_tsquery` for user input** — It never throws errors and supports familiar Google-style syntax.
3. **GIN over GiST for read-heavy** — GIN is faster for searches; GiST is faster to update. Choose based on read:write ratio.
4. **Weighted search** — Always use weight classes (A/B/C/D) for title/body separation to get better relevance ranking.
5. **`ts_rank_cd` over `ts_rank`** — Cover-density ranking generally produces better results for longer documents.
6. **Combine with B-tree conditions** — Add `published = true` or `created_at > now() - interval '1 year'` before the FTS condition to allow partial index scans.

Performance Considerations

```
-- Compare search approaches (scale to millions of rows first)
-- BAD: LIKE scan, no index, O(n)
```

```

EXPLAIN SELECT * FROM articles WHERE body LIKE '%postgresql%';

-- GOOD: GIN index lookup, O(log n + matches)
EXPLAIN SELECT * FROM articles WHERE search_vector @@ plainto_tsquery('postgresql');

-- Profile the query
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT title, ts_rank_cd(search_vector, q) AS r
FROM articles, websearch_to_tsquery('english', 'postgresql index') q
WHERE search_vector @@ q
ORDER BY r DESC LIMIT 10;

-- tune GIN index pending list (high-write tables)
-- in postgresql.conf:
-- gin_pending_list_limit = 4MB (default 4MB, increase for write-heavy)

```

Rule of thumb: For tables > 100K rows, GIN-indexed FTS is 100-1000x faster than `ILIKE '%term%'` .

Interview Questions & Answers

Q1: What is a `tsvector` and how is it created?

A: A `tsvector` is a pre-processed, sorted list of lexemes — normalized word forms with optional position numbers and weights. It's created by `to_tsvector(config, text)` , which tokenizes the text, applies the specified language's dictionary (for stop words and stemming), and stores the resulting normalized terms. For example, `to_tsvector('english', 'running fast')` produces `'fast':2 'run':1` — "running" is stemmed to "run".

Q2: What is the difference between `to_tsquery` and `websearch_to_tsquery` ?

A: `to_tsquery` expects input already in tsquery syntax (`word & word` , `word | word`) and throws an error on invalid syntax. `websearch_to_tsquery` accepts natural Google-style input (space = AND, `-` = NOT, `OR` keyword, "quotes" for phrases) and never throws errors. `websearch_to_tsquery` is recommended for user-facing search; `to_tsquery` for programmatic queries.

Q3: When would you choose GiST over GIN for full-text search?

A: GiST is preferred when the table has very high write frequency (many updates/inserts) because GiST indexes update cheaply. GiST can also index multiple column types in a single index. GIN is preferred for read-heavy workloads — it provides exact matches (no false positives/recheck needed) and is significantly faster for lookups. For most production use cases, GIN is the right choice.

Q4: Explain how `ts_rank_cd` (cover density) differs from `ts_rank` .

A: `ts_rank` counts term frequency — how often query terms appear in the document. `ts_rank_cd` measures cover density — how densely the query terms are packed together in the document. Documents where query terms appear close together score higher with `ts_rank_cd` . For multi-word queries, `ts_rank_cd` usually produces more intuitive relevance scores.

Q5: How do you implement a weighted search where title matches count more than body matches?

A: Use `setweight()` to assign weight classes before concatenating vectors:

```
setweight(to_tsvector('english', title), 'A') || setweight(to_tsvector('english',
```

body), 'C') . Then pass a custom weight array to `ts_rank` : `ts_rank('{0.1, 0.2, 0.4, 1.0}', vector, query)` where the array values correspond to weights D, C, B, A respectively.

Q6: How do you handle NULL values in full-text search columns?

A: Use `COALESCE` : `to_tsvector('english', COALESCE(subtitle, '')) || ' ' || COALESCE(body, ''))` . NULL concatenated with text produces NULL, which would make the entire tsvector NULL and cause that row to never match. Alternatively, use a CASE expression or separate vector concatenation:
`COALESCE(to_tsvector('english', subtitle), '')` .

Q7: What is a language configuration in PostgreSQL FTS?

A: A text search configuration (e.g., `english` , `french` , `simple`) defines how text is parsed and normalized. It specifies which parser to use, which dictionaries apply to which token types, and how stop words are handled. `simple` does no stemming — all tokens are lowercased only. `english` applies Porter stemming and English stop words. Using the wrong configuration degrades search quality.

Q8: How does phrase search work with `<->` ?

A: The `<->` operator in tsquery matches when two lexemes appear adjacent in the document (consecutive positions). `'full' <-> 'text'` matches "full text" but not "full database text". The `<N>` variant allows a gap: `'postgres' <3> 'guide'` matches if "guide" appears within 3 words after "postgres". This requires position information, so the tsvector must be created with positions (default for `to_tsvector`).

Q9: How would you implement auto-suggest/autocomplete using FTS?

A: Use prefix matching with `:*` in tsquery: `to_tsquery('english', 'postgr:*')` matches any term starting with "postgr". Combine with `ts_rank` for relevance. For real autocomplete (substring from start), you'd also consider `pg_trgm` for trigram-based prefix searches, which can be faster for short-prefix queries.

Q10: What are the performance implications of using `ts_headline` ?

A: `ts_headline` is expensive — it re-parses the original text and scans it for matches. It does NOT use indexes. Therefore, always filter and paginate FIRST (using the indexed `search_vector @@ query` condition), then apply `ts_headline` only to the final result set (typically 10-20 rows). Never call `ts_headline` in a subquery or aggregate over a large result set.

Q11: How do you keep a stored `tsvector` column in sync?

A: Two approaches: (1) Trigger — `CREATE TRIGGER ... BEFORE INSERT OR UPDATE ... EXECUTE FUNCTION update_search_vector()` . The trigger fires on every INSERT/UPDATE and recalculates the vector. (2) Generated column (PostgreSQL 12+) — `GENERATED ALWAYS AS (...) STORED` . This is simpler and atomic but less flexible (can't call session-dependent functions). Both require a GIN index on the resulting column.

Q12: How do you search across multiple tables (e.g., products, articles, users)?

A: Use a `UNION ALL` query across tables: each subquery searches its own table with FTS and returns a common result schema including a `source_type` column. Apply ranking in the outer query. For high-performance cross-table search, consider a dedicated search table or materialized view that aggregates FTS vectors from multiple tables, or use a search platform like Elasticsearch for complex cross-entity search.

Exercises and Solutions

Exercise 1

Add a `search_vector` column to the `articles` table, auto-populated by a trigger. Test that inserting a new article automatically populates the vector.

Solution:

```
-- Trigger already shown above – test it:
INSERT INTO articles (title, body, author, published)
VALUES ('Trigger Test', 'Testing automatic vector population', 'Test User', true);

SELECT title, search_vector IS NOT NULL AS has_vector
FROM articles WHERE title = 'Trigger Test';
-- Should return: has_vector = true
```

Exercise 2

Write a query that returns the top 5 articles matching "database optimization", ranked by relevance, with a highlighted excerpt from the body (max 2 fragments, 30 words each).

Solution:

```
SELECT
  a.title,
  ts_rank_cd(a.search_vector, q, 32) AS relevance,
  ts_headline('english', a.body, q,
    'MaxFragments=2, MaxWords=30, MinWords=10, StartSel=>>>, StopSel=<<<')
  AS excerpt
FROM articles a,
  websearch_to_tsquery('english', 'database optimization') AS q
WHERE a.search_vector @@ q
ORDER BY relevance DESC
LIMIT 5;
```

Exercise 3

Write a query using `ts_debug` to show how the text "PostgreSQL Full-Text Search running fast" is parsed and normalized using the 'english' configuration.

Solution:

```
SELECT alias, description, token, dictionaries, dictionary, lexemes
FROM ts_debug('english', 'PostgreSQL Full-Text Search running fast');
-- Shows each token, its type, which dictionary was used, and the resulting lexeme
```

Cross-References

- [01_jsonb_mastery.md](#) — GIN indexes also used for JSONB; same index structure
- [07_indexes](#) — B-tree vs GIN vs GiST index architecture
- [05_extensions_overview.md](#) — `pg_trgm` extension for trigram similarity search

- **04_materialized_views.md** — Materializing search vectors from multiple tables
- **09_stored_procedures_functions.md** — Trigger implementations
- **PostgreSQL Docs** — <https://www.postgresql.org/docs/current/textsearch.html>